

Research question

"How many robots do we need to complete the warehouse workload on time, without collisions or congestion, and with minimal energy use?"

too few robots means slow completion, too many means congestion and wasted energy.

Robot design

One robot type with identical physical parameters. All robots have the same speed, battery capacity, sensor range, and turn radius. The only difference between robots is their assigned algorithm. This isolates the algorithm as the single variable in experiments and keeps us safe on project requirements.

Robot states: Idle, ToPickup, Loading, ToDropoff, Unloading, Charging

Energy model: Battery drains based on distance traveled, turning, and carrying load. If below threshold, robot must go charge. If battery hits zero, robot is disabled and logged as a failure.

Task system

Core task is pick and deliver. A pick request appears with a shelf location and packing station destination. Robot travels to shelf, spends N ticks loading, travels to packing station, spends M ticks unloading.

Task assignment is centralized. The dispatcher assigns tasks to robots based on availability, proximity, and battery level. Robots do not self-discover tasks. This keeps separation of concerns clean and makes fairness easier to manage.

Stretch goals: task priorities, time limits per task, replenishment tasks from dock to shelf.

Map system

Tile types: wall, shelf (obstacle), aisle (walkable), packing station, dock, charger

Map modes:

- Preset maps for MVP: small baseline for demos, narrow aisles stress test, many intersections stress test
- Randomized maps: parameters for size, shelf density, station count, plus seed for reproducibility
- Custom editor as stretch goal: place tiles manually, validate connectivity

Decision-making algorithms

Navigation strategies (one assigned per robot):

- Greedy: move toward goal, pick next-best direction if blocked. Simple baseline.
- Wall-follow / Bug: follow obstacle boundary until goal direction is clear again. Good for partial observability.
- Real-time A*: pathfind on known tiles only, treat unknown as high cost, replan when blocked. Most sophisticated

Coordination policies (system-wide, one selected per experiment):

- Traffic rules: priority at intersections (loaded robots yield to empty or vice versa), keep-right convention, don't enter occupied tiles
- Reservation-based: robots reserve their next k steps, prevents two robots claiming same tile at same tick

This gives us 3 navigation strategies times 2 coordination policies equals 6 experiment configurations.

Metrics

Throughput:

- Tasks completed per minute
- Average delivery time per task
- Idle time per robot

Congestion:

- Collision count
- Near-miss count
- Time spent waiting or not moving
- Path heatmaps showing which tiles get congested

Energy:

- Total energy consumed
- Energy per task (efficiency measure)
- Wasted energy from detours or failed paths
- Robots disabled from battery death

Fairness:

- Task distribution across robots
- Utilization per robot

Recovery:

- Time to recover after collision or blockage
- Deadlock occurrences

Deadlock handling

Detection: if a robot makes no progress for T ticks while not loading, unloading, or charging, flag as stuck.

Resolution: timeout causes task reassignment to another robot, stuck robot returns to idle or goes to charger.

All deadlock events logged for analysis.

Database (PostgreSQL)

Maps table: id, name, dimensions, tile data, preset flag, random seed

Simulation runs table: id, map id, robot count, algorithm configuration, coordination policy, workload seed, timestamp

Results table: run id, throughput, collisions, near misses, energy, completion time, deadlock count

Simulation logs table: run id, tick, robot id, event type, position, details. Events include task assigned, task completed, collision, near miss, charging start, deadlock, battery death.

Robot stats table: run id, robot id, tasks completed, distance traveled, energy used, idle time, collision count

Config file (JSON)

Contains: selected map, list of robots with their navigation algorithm, coordination policy for the run, workload settings (task count, spawn rate, seed), simulation settings (max ticks, speed multiplier)

Module breakdown

Simulation Engine (Lead): tick loop, time control, event bus, start/pause/stop, collision resolution after collecting move intentions

Map (Dev 1): tile grid, preset maps, random generator, validation

Robot Core (Dev 2): robot state machine, energy model, movement execution

Sensors (Dev 3): vision calculation, fog-of-war discovery, local map memory per robot

Algorithms (Dev 4): navigation strategies, pathfinding, replanning logic

Dispatcher (Dev 5): task generation, task assignment, coordination policy enforcement, deadlock detection

UI / JavaFX (Dev 6): screens, rendering, controls, heatmaps, stats dashboard

Database + Config (QA): PostgreSQL layer, JSON loader/saver, logging, metrics calculation

Algorithm module interfaces (for my part specifically)

Receives: robot current position, robot known map from sensors, assigned target from dispatcher, reservation or traffic state depending on coordination policy

Returns: next move intention (direction plus action type)

Does not handle: collision resolution (engine does this), actual position updates (robot core), what the robot can see (sensors)

Open questions

Should coordination policy be per-experiment only or can robots mix policies in one run?

Per-experiment

How long is a tick in real time?

Who owns the reservation table, dispatcher or engine? **Engine**

MVP:

P0 - Basic GUI (welcome, simulation, exit)

P0 - Sensors (knowing whats around)

P0 - Simple decision-making algorithms

P0 - Greedy-style dispatching